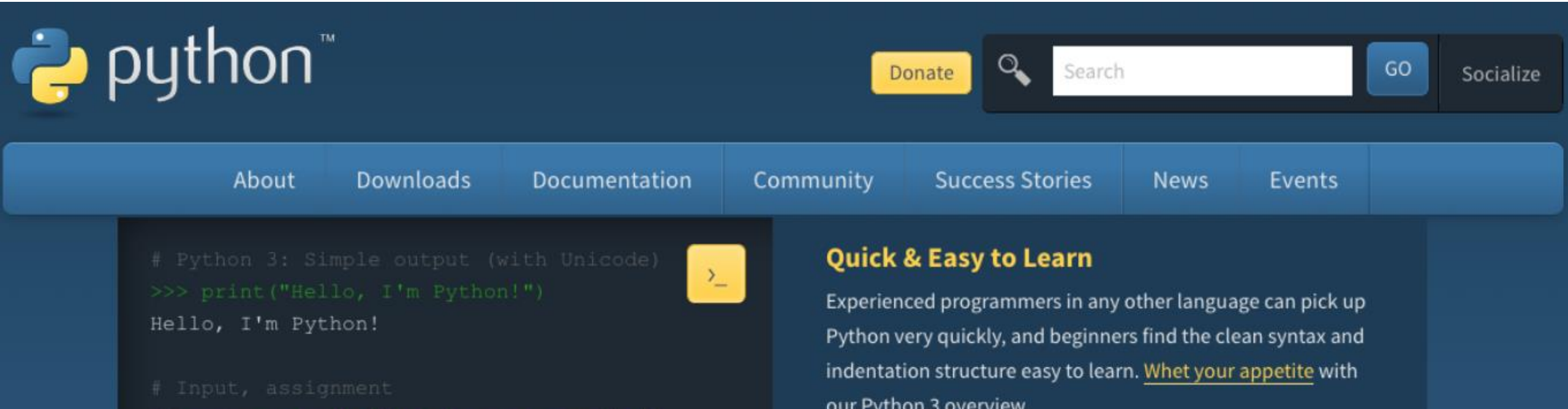
Welcome to MA3K7
Problem Solving with Python
Live online lab – Week 3



This session is being recorded

What we'll cover in this session

- Scope of variables in Python
- Simple sliders in Python
- Reminders about
  - linear algebra
  - histograms, bar charts
- Hadamard's maximum determinant problem.
- Python functions for solving matrix equations

Check your version of Python

```
[1]: from platform import python_version

     python_version()

[1]: '3.9.12'
```

Current version is 3.12.

Your version should be at least 3.9.

Variable scope

•[66]:
```
a = 7

def my_func():
    a = 6
    print(a)

print(a)

my_func()
```

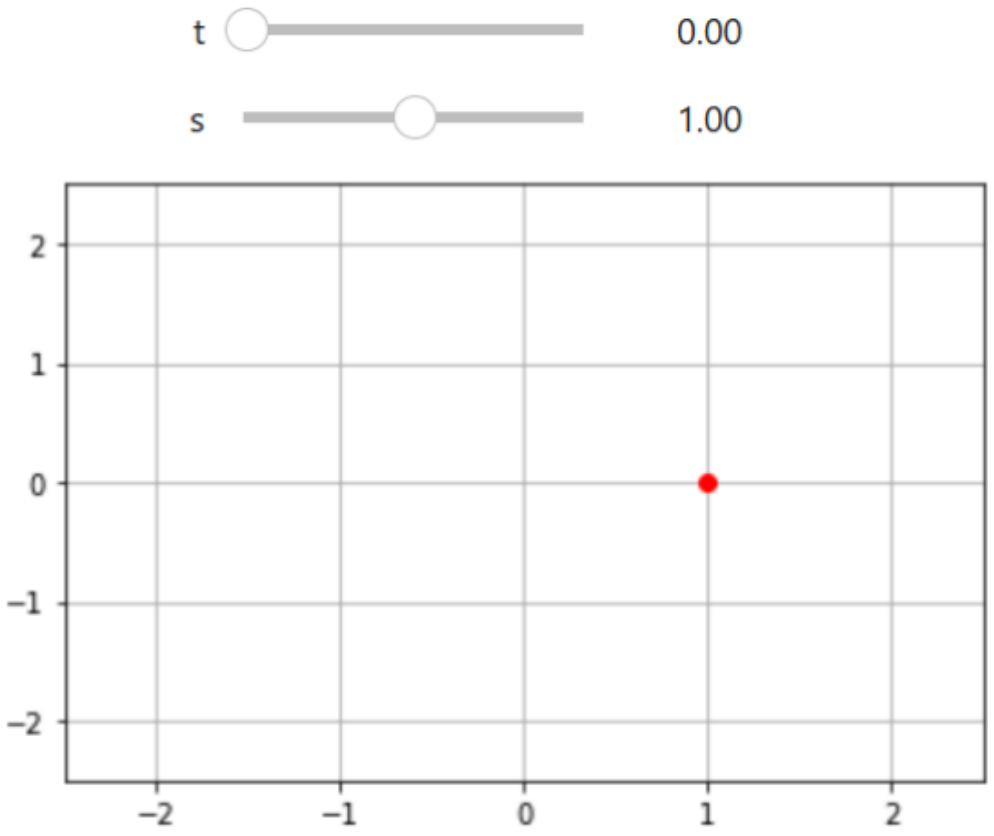
Two variables called a, one inside the function, one outside operating independently.

•[75]:
```
a = 7

def my_func():
    if a == 7:
        print("I can see a")
        print("and I can print it:",a)

my_func()
```

If a function doesn’t declare a variable with the same name as a variable in the main program it can read that variable fro the main program

Simple implementation of sliders with **interactive**

[70]:
```
import numpy as np
import matplotlib.pyplot as plt
from ipywidgets import interactive

def plot(t = 0.0, s = 1.0):
    plt.plot(s*np.cos(t),s*np.sin(t),"ro")
    plt.xlim(-2.5,2.5)
    plt.ylim(-2.5,2.5)
    plt.grid()
    plt.show()

interactive(plot,t = (0,6.28,0.01),s = (0,2,0.01))
```



Put your plotting into a function whose inputs are the values to be controlled by sliders

Use interactive( ) as in this example to set up the sliders, this one has two sliders

Linear Algebra – Siri's Book

Page 217

5.2 *Basic concepts in linear algebra*

All code snippets in this section are to begin with the following lines:

```
import numpy as np
import scipy.linalg as LA
import sympy as sp
```

Matrices and basic operations	
Defining a matrix	
Numerical : $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	`A = np.array([[1,2], [3,4]])`
Symbolic : $B = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$	`a,b,c,d = sp.symbols("a,b,c,d")` `B = sp.Matrix([[a,b], [c,d]])`
Identity and zero matrices	
$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ and $Z = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$	
Numerical	`I = np.eye(2)` `Z = np.zeros((2,3))`
Symbolic	`I = sp.eye(2)` `Z = sp.zeros(2,3)`
Multiplying two matrices	
Numerical: A^2 (Be careful! Both `A*A` and `A**2` mean squaring elementwise)	`A@A`
Symbolic: B^2	`B*B  # or B**2 or B@B`
Transposing	
Numerical: A^T	`A.T  # or np.transpose(A)`
Symbolic: B^T	`B.T`
Finding the determinant	
Numerical: $\det A = -2$	`LA.det(A)`
Symbolic: $\det B = ad - bc$	`B.det()`
Finding the inverse	
Numerical: A^{-1}	`LA.inv(A)`
Symbolic: B^{-1}	`B**-1`

Linear algebra

•[340]:

```
import numpy as np

A = np.arange(10) # A is 1D array with entries 0 to 9 inclusive

A = A.reshape(2,5) # A is reshaped as a 2 x 5 matrix

B = np.zeros_like(A) # B would then be a 2 x 5 matrix with all entries zero

C = np.ones_like(B) # C would then be a 2 x 5 matrix with all entries one

D = np.zeros((2,3)) #  a 2 x 3 matrix with all entries zero

E = np.random.random((5,2)) # a 2 x 3 matrix with all entries a random number between 0 and 1

F = np.matmul(A,E) # matrix product AE

G = np.identity(2)

H = np.ones_like(G)

print(np.matmul(H,H)) #matrix multiplication
print(H*H) #elementwise multiplication
print((2*H)**3) #elementwise power not the cube of the matrix 2H
```

Random Values Siri's Book

Page 362

	NumPy method	Python method
Pick a random number in [0,1)	`rng = np.random.default_rng()` `rng.random()`	`import random` `random.random()`
Pick N random numbers in [0,1)	`rng.random(N)` `# An array`	`[random.random() for i in range(N)]` `# A list`
A matrix of random numbers	`rng.random((M,N))` `# An MxN array`	

Pick a random number from a uniform distribution $U(a, b)$	`# Result in [a,b)`
	`a + (b-a)*rng.random()`
	`# OR`
	`rng.uniform(a,b)`
		`# Result in [a,b]`
		`random.uniform(a,b)`

Pick a random integer between m and n	`# Result in [m,n)`
	`rng.integers(m,n)`
		`# Result in [m,n)`
		`random.randrange(m,n)`
		`# OR, for result in [m,n]`
		`random.randint(m,n)`

Random numbers in Python with numpy

```
•[191]: import numpy as np

rdm01 = np.random.random() # random value in [0,1)

rdm1darray = np.random.random(5) #array of length 5 with random values in [0,1)

rdm2darray = np.random.random((2,4)) #a matrix with random entries in [0,1)

rdmab = np.random.uniform(2,5) # gives a random value in (2,5)

rdintab = np.random.randint(2,5) # gives a random integer in [2,5)
```

Generating random values using **np.random**

Histograms, with frequency

•[195]:
```
import numpy as np
import matplotlib.pyplot as plt
import random

data = [1,2,2,3,3,3,4,4,4,4,5,5,5,5,5] # the data, can be passed as array

mybins = [0.5,1.5,2.5,3.5,4.5,5.5] # the endpoints of the 'containers' can be passed as array

freq,bins,obj = plt.hist(data,bins = mybins)
# freq, bins, obj returns the array of frequencies, the bins and the data type

plt.show()
```

Plotting a histogram in matplotlib, frequency on y-axis

Histograms, with probability

•[200]:
```
import numpy as np
import matplotlib.pyplot as plt
import random

data = [1,2,2,3,3,3,4,4,4,4,5,5,5,5,5] # the data, can be passed as array

mybins = [0.5,1.5, 2.5,3.5,4.5,5.5] # the endpoints of the 'containers' can be passed as array

prob,bins,obj = plt.hist(data,bins = mybins, density = True)
# prob, bins, obj returns the array of probabilities (but only if all bins have width 1)
# the bins and the data type

plt.show()
```

Plotting a histogram in matplotlib, frequency on y-axis

Siri's Book: binomial distribution

Page 372

coin1.ipynb (for plotting fig. 7.6 and calculating Pr(at least 5 heads))

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import random`
	`%matplotlib`

Number of 10-throw experiments to simulate	`exprmt = int(1e5)` `throws = 10`
This will be filled with heads counts	`Htally = []`
Throw the coin lots of times	`for i in range(exprmt):` `  heads= 0`

Use **Method 1** to simulate throws	`  for trial in range(throws):`
	`  roll = random.randint(0, 1)`
	`  if roll == 0: heads += 1`

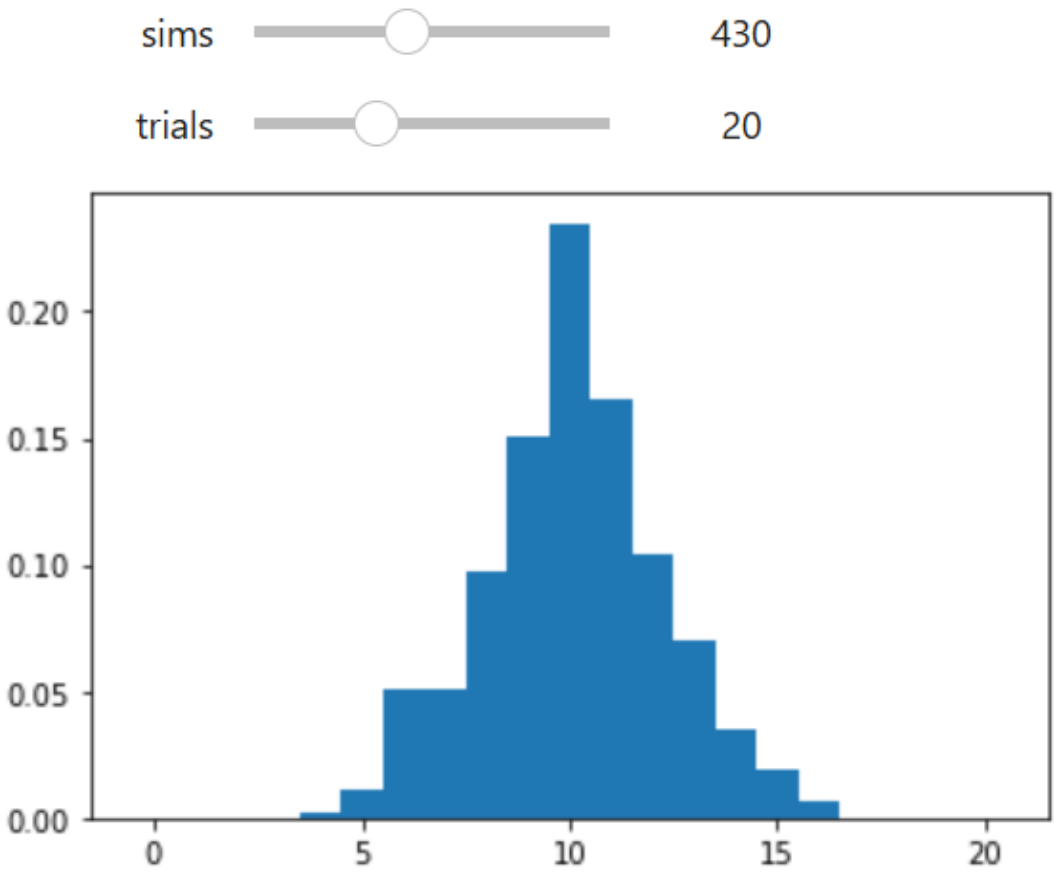
Record the number of heads	`  Htally.append(heads)`
Plot 11 bars (centred on integers)	`bins = np.arange(throws+2)-0.5`

Plot histogram (prob=bar height) density=True normalises the distribution	`prob, b1, b2 = plt.hist(Htally, bins= bins,`
	`  density = True)`
	`plt.xticks(range(11))`
	`plt.xlabel('No. of heads in 10 throws')`
	`plt.ylabel('Probability')`
	`plt.grid('on')`
	`plt.show()`

Obtain probabilities from histogram	`print(f'Pr(5 heads)= {prob[5]}')` `print(f'Pr(at least 5 heads)= {sum(prob[5:])}')`

Binomial with slider

```
[270]: #binomial with slider
import numpy as np
import random as rnd
import matplotlib.pyplot as plt
from ipywidgets import interactive

def bin(sims = 200,trials = 10):
    simres=[]
    for i in range(sims):
        heads = 0
        for j in range(trials):
            if np.random.random()<0.5:
                heads += 1
        simres.append(heads)
    mybins = np.arange(0,trials+2)-0.5
    plt.hist(simres,bins = mybins, density = True)
    plt.show()

interactive(bin, trials = (5,50,1), sims =(10, 1000, 10))
```



Hadamard's maximum determinant problem

1 An interesting determinant problem

Let A be a 3×3 matrix whose entries are randomly chosen binary numbers (*i.e.* 0 or 1).

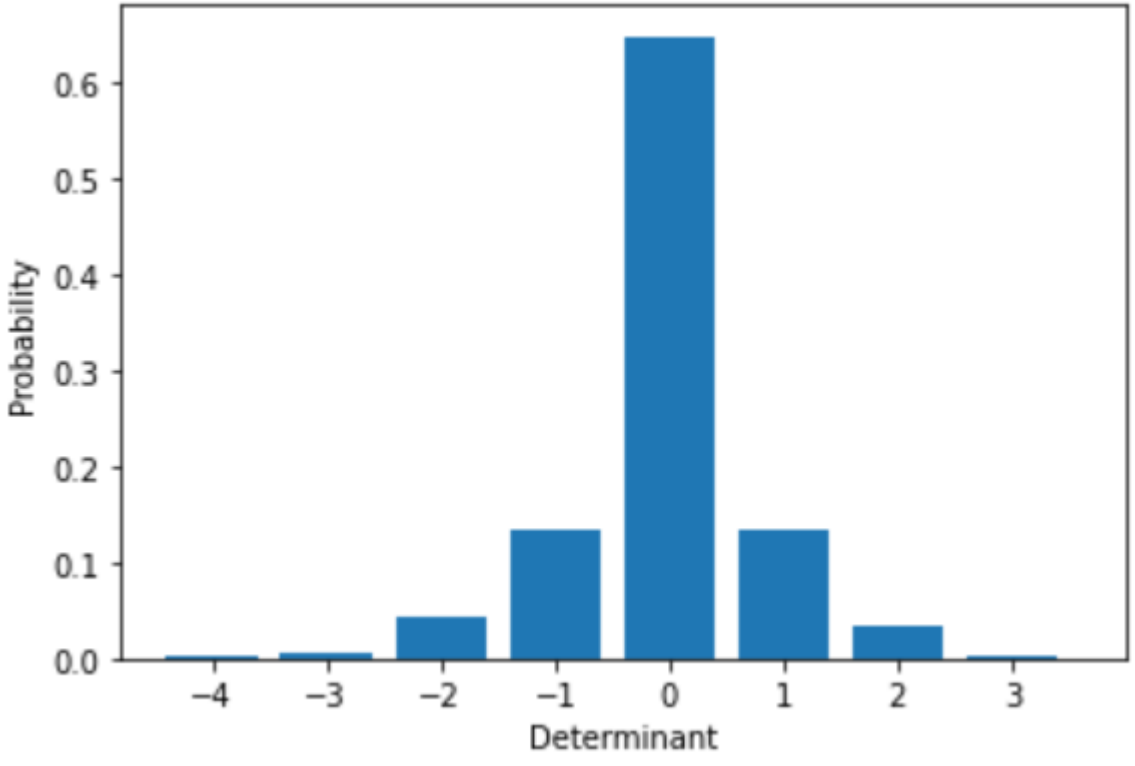
a) What values can $\det A$ take?

b) What is the probability that $\det A = 0$?

c) Investigate what happens when the size of A increases.

Hadamard's determinant problem

```
1]: import numpy as np
import matplotlib.pyplot as plt
import random as rdm

trials = 1000

dim = 5

results = []

for trial in range(trials):
    M = np.round(np.random.random(dim**2).reshape(dim,dim))
    det = np.linalg.det(M)
    results.append(det)

mybins = np.arange(min(results),max(results)+2)-0.4

plt.hist(results, bins=mybins, density = True, width = 0.8)
plt.xticks(np.arange(min(results),max(results)+1))
plt.xlabel("Determinant")
plt.ylabel("Probability")
plt.show()
```



Linear algebra – solving matrix equations

2 Timing your code

Let A be a $n \times n$ matrix filled with random small numbers between 0 and 1.
Let $\mathbf{b} = (1, 1, 1, \ldots 1)^T$ (where there are n entries).
Use Python to solve the system

$$A\mathbf{x} = \mathbf{b}.$$

How long does it take? How does the runtime increase as n increases to, say, 2000.

We can solve the linear system in two ways.

First, we can obviously write $\mathbf{x} = A^{-1}\mathbf{b}$. In Python, we use the syntax

```
x = scipy.linalg.inv(A)*b
```

Alternatively, we can avoid inverting A, and perform what's called LU decomposition on A. This gives X without explicitly finding the inverse. To do this in Python, we use the syntax

```
x = scipy.linalg.solve(A,b)
```

To find out how the runtimes for these two methods compare, download `solvetimes.ipynb` from my GitHub page[1]. We'll discuss the code in class.

More about this in Section 5.4 in the book.

Linear Algebra – solving matrix equations

•[61]:

```
import numpy as np
import time as t
import scipy.linalg

size = 100

#with matrix inverse
start = t.time()
for i in range(1000):
    A = np.random.random((size,size))
    b = np.ones((size,1))
    x = scipy.linalg.inv(A)@b
stop = t.time()
print(stop-start)

# with LU decomposition

start = t.time()
for i in range(1000):
    A = np.random.random((size,size))
    b = np.ones((size,1))
    y = scipy.linalg.solve(A,b)
stop = t.time()
print(stop-start)
```

Module sessions

- Mondays, 2pm-3pm, online through Teams - exploring the use of Python

- Tuesdays, 1pm-3pm, OC0.02 (Oculus) - interactive workshop class